

ECMA-55 BASIC

Manual da Linguagem

Augusto Manzano

2026

Este manual documenta a linguagem BASIC exatamente como definida pela norma ECMA-55 (1ª edição, janeiro de 1978). A mesma norma que a sigla ANSI X3.60-1978 descreve, conhecida informalmente como “Minimal BASIC”.

O interpretador (`ecma55basic`) implementa só o que a norma define, sem nenhum acréscimo. É um material didático para conhecer o BASIC como ele nasceu, antes de cada fabricante acrescentar suas próprias extensões.

Sumário

1. Visão geral
2. Elementos léxicos
3. Variáveis e arrays
4. Expressões e operadores
5. Funções embutidas
6. Referência de comandos
7. Erros e exceções
8. O ambiente interativo

Como usar este manual

Se você nunca programou em BASIC, leia os capítulos na ordem. Se você já conhece a linguagem (ou uma variante dela, como o GW-BASIC ou o QBasic) e só quer conferir uma sintaxe específica, vá direto ao capítulo 6 (comandos) ou 5 (funções).

Todo exemplo de código neste manual roda de verdade neste interpretador. Teste você mesmo, escrevendo `ecma55basic seu-programa.bas` ou colando o código no ambiente interativo (capítulo 8).

1. Visão geral

1.1. O que é o Minimal BASIC

Em 1978, quando o BASIC já rodava em dezenas de computadores diferentes cada fabricante com sua própria versão, incompatível com as demais duas entidades de padronização, a ANSI (americana) e a ECMA (europeia), publicaram uma norma comum descrevendo o núcleo mínimo que qualquer implementação de BASIC deveria oferecer para poder se chamar, de fato, "BASIC". Essa norma é a ECMA-55 (idêntica, palavra por palavra, à ANSI X3.60-1978).

"Mínimo" aqui é o ponto central: a norma não tentou descrever o BASIC de nenhum fabricante específico, mas sim o menor subconjunto comum a todos eles, um alicerce sobre o qual cada fabricante poderia construir suas próprias extensões sem deixar de ser, tecnicamente, compatível com a norma. Por isso a linguagem é conhecida informalmente como "Minimal BASIC".

Isso explica por que a linguagem, vista com olhos de hoje, parece tão enxuta: sem funções de string além das mínimas, sem WHILE, sem registros, sem tipos definidos pelo programador. Não é uma limitação de projeto malfeito, é a definição deliberada de um piso comum.

1.2. Por que um interpretador estritamente aderente à norma

Este projeto implementa exatamente o que a norma define, e nada além disso. Tem como objetivo servir de material didático fiel ao que o BASIC *era*, historicamente, antes de cada fabricante acrescentar o que quis. Um estudante que aprende com este interpretador aprende a norma, não uma variante dela.

1.3. Compilando e executando

O projeto usa um Makefile simples, dentro da pasta do projeto.

Linux

```
make          - compila o interpretador (gera o executável ecma55basic)
make test     - compila e roda a suíte de testes própria (lexer, parser, interpretador)
make nbs      - compila e roda a bateria de conformidade da NBS
make clean    - remove os arquivos gerados pela compilação
```

Windows

```
mingw32-make          - compila o interpretador (gera o executável ecma55basic)
mingw32-make test     - compila e roda a suíte de testes própria (lexer, parser, interpretador)
mingw32-make nbs      - compila e roda a bateria de conformidade da NBS
mingw32-make clean    - remove os arquivos gerados pela compilação
```

Requer um compilador C++17 (o g++, por exemplo) e o utilitário make/mingw32-make. Nenhuma outra dependência.

Depois de compilado, há duas formas de usar o interpretador:

```
./ecma55basic          # abre o ambiente interativo (ver capítulo 8)
./ecma55basic programa.bas  # executa um arquivo de programa direto
```

1.4. A estrutura de um programa

Um programa em Minimal BASIC é uma sequência de linhas numeradas, uma abaixo da outra, em ordem estritamente crescente de número. Cada linha contém exatamente um comando. A última linha do programa, e somente ela, precisa ser um comando END.

```
10 PRINT "OLA, MUNDO"
20 END
```

O número de linha (aqui, 10 e 20) não é decoração: é ele que define a ordem de execução do programa (que roda, por padrão, na ordem numérica das linhas) e é ele que os comandos de desvio (GOTO, GOSUB, IF . . . THEN) usam para indicar para onde pular. Um número de linha pode ter de 1 a 4 dígitos (de 1 a 9999).

Por convenção, não por exigência da norma, a maioria dos programas numera as linhas de 10 em 10 (10, 20, 30...), deixando espaço para inserir novas linhas no meio depois, sem precisar renumerar tudo.

1.5. Maiúsculas e minúsculas

A norma define que a linguagem BASIC é escrita inteiramente em letras maiúsculas: palavras reservadas, nomes de variáveis, tudo. Este interpretador aceita minúsculas no código-fonte como uma conveniência prática e as normaliza para maiúsculas internamente, mas o conteúdo de uma cadeia entre aspas ("Assim") preserva exatamente as maiúsculas e minúsculas que você escreveu. Afinal, ali é dado, não código.

1.6. O que vem a seguir

O capítulo 2 detalha o “alfabeto” da linguagem, como um número, uma cadeia ou um nome de variável são reconhecidos. Se você já tem familiaridade com a sintaxe geral do BASIC, pode pular direto para o capítulo 6 (referência de comandos).

2. Elementos léxicos

Este capítulo descreve as menores unidades que compõem um programa: o “alfabeto” da linguagem. Antes de um comando fazer sentido, o interpretador precisa reconhecer, caractere por caractere, onde termina um número e começa uma palavra reservada, onde termina uma cadeia e começa outra coisa. É esse reconhecimento que este capítulo descreve.

2.1. Conjunto de caracteres

A norma define a linguagem sobre um alfabeto de letras maiúsculas (A–Z), dígitos (0–9) e um conjunto de símbolos de pontuação e operadores. Este interpretador aceita minúsculas no código-fonte como conveniência prática (normalizadas para maiúsculas), mas o conteúdo de uma cadeia entre aspas preserva exatamente o que foi escrito: maiúsculas, minúsculas, espaços, tudo.

Cada linha de um programa tem no máximo 72 caracteres.

2.2. Números

Um número é escrito em notação decimal, com um ponto (não vírgula) separando a parte inteira da fracionária, e um expoente opcional introduzido por E:

```
500
-21.
.255
1E10
5E-1
.4E+1
```

Repare que -21. é válido (o ponto sem nada depois), assim como .255 (sem nada antes do ponto). O sinal de um número, quando aparece isolado na frente de uma expressão ou depois de um operador, é tratado como parte da expressão e não do número em si.

Internamente, os números são armazenados em ponto flutuante de dupla precisão (o tipo `double` do C++), o que garante bem mais dígitos de precisão do que qualquer programa Minimal BASIC deveria precisar.

2.3. Cadeias

Uma cadeia é um texto entre aspas duplas:

```
"OLA, MUNDO"
"Pode ter minúsculas, números 123, e pontuação!"
```

Não existe mecanismo de escape, uma cadeia não pode conter uma aspa dentro dela, porque a primeira aspa que aparecer fecha a cadeia. Não há limite de tamanho para uma cadeia *constante* (escrita direto no código), mas uma variável de cadeia (seção 3.2) tem um mínimo garantido de 18 caracteres.

2.4. Identificadores (nomes)

A norma define nomes bem restritos, de forma bem diferente das linguagens modernas:

Forma	Significa	Exemplos
Uma letra	Variável numérica simples	A, X, Z
Uma letra + um dígito	Variável numérica simples	A1, X9, B0
Uma letra + \$	Variável de string	A\$, N\$
FN + uma letra	Função definida com DEF FN (capítulo 4)	FNA, FNF

Não existem nomes descritivos como TOTAL ou CONTADOR, o máximo que a linguagem permite é uma letra e, opcionalmente, um dígito ou o \$. É uma limitação real da norma, não deste interpretador: programe com nomes curtos, como se fazia na época.

Um arranjo (capítulo 3) usa como nome uma letra sozinha, nunca letra+dígito. DIM A9(10) é inválido; DIM A(10) é válido.

2.5. Palavras reservadas

As 24 palavras a seguir têm significado fixo na linguagem e não podem ser usadas como nome de variável (o que, dado que nomes de variável têm no máximo duas letras/dígitos, nunca seria um conflito de qualquer forma):

```
BASE DATA DEF DIM END FOR GO IF INPUT LET
NEXT ON OPTION PRINT RANDOMIZE READ REM RESTORE RETURN
STEP STOP SUB THEN TO
```

GOTO e GOSUB não são palavras à parte, são a grafia colada de GO TO e GO SUB (a norma permite escrever com ou sem espaço entre GO e TO/SUB). As onze funções embutidas (capítulo 5) e o TAB do PRINT também são reservados, mas diferente das 24 palavras acima. Podem aparecer colados a um parêntese, sem espaço obrigatório antes (SQR(4), TAB(10)).

Toda palavra reservada (das 24 principais) precisa ter um espaço antes dela e, se não for o fim da linha, um espaço depois. 100LET X=5 é inválido; precisa ser 100 LET X=5.

2.6. Comentários

O comando REM (capítulo 6) marca o resto da linha como comentário: texto livre, ignorado pelo interpretador:

```
100 REM ESTE PROGRAMA CALCULA A MEDIA DE TRES NOTAS
```

Não existe comentário de fim de linha (tipo ' ou //) depois de um comando, REM precisa ser o comando inteiro da linha.

2.7. Operadores e pontuação

+ - * / ^ operadores aritméticos (capítulo 4)
= <> < <= > >= operadores relacionais (capítulo 4)
() parênteses de expressão, array e função
, ; separadores (PRINT, listas de variáveis, DIM)

O capítulo 4 explica a precedência entre os operadores aritméticos e como eles se comportam.

3. Variáveis e arranjos

3.1. Dois tipos, só dois

A linguagem tem exatamente dois tipos de dado: **número** (ponto flutuante) e **cadeia** (texto). Não existem inteiros como tipo separado, não existem booleanos (uma comparação como $X > 3$ produz um número: 1 para verdadeiro, 0 para falso), não existem tipos definidos pelo programador.

O tipo de uma variável é determinado pela sua própria grafia. A é sempre numérica, A\$ é sempre cadeia, nunca pelo valor que você atribui a ela.

3.2. Variáveis numéricas e de cadeia

Uma variável numérica simples é uma letra, opcionalmente seguida de um dígito (A, B3, Z9, veja a seção 2.4). Uma variável de cadeia é uma letra seguida de \$ (A\$, N\$).

```
100 LET X = 5
110 LET N$ = "MARIA"
```

Toda variável precisa receber um valor antes de ser lida. Ler uma variável que nunca foi atribuída é um erro fatal, que interrompe o programa (capítulo 7). Decisão de projeto que segue a recomendação explícita da norma (seção 7.6), em vez do costume comum de assumir zero silenciosamente. Isso é deliberado: um valor “esquecido” costuma ser sinal de um bug no programa, e é melhor descobrir isso na hora do que carregar um zero incorreto adiante sem perceber.

Uma cadeia armazenada numa variável tem um mínimo garantido de 18 caracteres. Tentar guardar uma cadeia maior é erro fatal com LET e READ, mas não com INPUT (que, nesse caso, simplesmente pede a resposta de novo, sem derrubar o programa).

3.3. Arranjos

Um arranjo é uma “tabela” de valores numéricos, indexada por um número inteiro (ou dois, para tabelas bidimensionais). O nome de um arranjo é sempre uma única letra, nunca letra+dígito.

Um arranjo não precisa ser declarado explicitamente: se você usar A(3) sem nunca ter dado um DIM A(. . .) antes, o arranjo A nasce automaticamente com índices de 0 a 10 (11 posições). Para um arranjo maior, ou para deixar o tamanho explícito no código, use DIM:

```
100 DIM A(100)
110 LET A(50) = 3.14
120 PRINT A(50)
```

Isso cria um arranjo A com índices de 0 a 100 (101 posições). DIM precisa vir antes do primeiro uso do arranjo, e um mesmo arranjo só pode ser dimensionado uma vez.

Um arranjo bidimensional usa dois índices, separados por vírgula:

```
100 DIM B(10, 20)
110 LET B(5, 12) = 7
```

Uma letra só pode ser uma coisa: variável simples, arranjo de uma dimensão, ou arranjo de duas dimensões. Nunca duas dessas ao mesmo tempo no mesmo programa. Usar A como variável simples numa linha e como A(3) noutra é erro.

Assim como as variáveis simples, um elemento de arranjo precisa receber um valor antes de ser lido.

3.4. OPTION BASE — mudando onde o índice começa

Por padrão, os índices de um arranjo começam em 0. O comando `OPTION BASE 1`, se usado, muda isso: os índices passam a começar em 1, e um arranjo sem DIM explícito nasce com índices de 1 a 10 (10 posições, não 11).

```
100 OPTION BASE 1
110 DIM A(10)
120 LET A(1) = 100
```

`OPTION BASE` só pode aparecer uma vez no programa, e precisa vir antes de qualquer DIM ou uso de arranjo, inclusive antes de um arranjo implicitamente criado pelo primeiro uso.

3.5. Onde as variáveis “moram”

Toda variável e todo arranjo são **globais**: visíveis de qualquer ponto do programa, do início ao fim. A única exceção é o parâmetro de uma função definida com `DEF FN` (capítulo 4), que é local àquela definição e não interfere com uma variável de mesmo nome usada no resto do programa.

4. Expressões e operadores

4.1. Operadores aritméticos e precedência

`^` potênciação
`*` / `/` multiplicação e divisão
`+` - `-` soma e subtração

De cima para baixo, da maior para a menor precedência exatamente como na matemática comum. Parênteses mudam a ordem normalmente:

```
PRINT 5 + 3 * 2
PRINT (5 + 3) * 2
```

A primeira linha imprime 11 ($3 \cdot 2$ primeiro, depois $+5$); a segunda imprime 16.

Um detalhe que surpreende quem já programou em outras linguagens: a potênciação (`^`) é associativa à **esquerda**, não à direita. Ou seja, A^B^C é calculado como $(A^B)^C$, não como $A^{(B^C)}$. Na maioria das linguagens modernas (Python, C, etc.) é o contrário — se você vem de lá, preste atenção nesse ponto.

```
PRINT 2^3^2
```

O resultado é $(2^3)^2 = 8^2 = 64$, **não** $2^{(3^2)} = 2^9 = 512$.

Operadores de mesma precedência (`*` com `/`, ou `+` com `-`) também são avaliados da esquerda para a direita: $10 - 3 - 2$ é $(10-3)-2 = 5$, não $10-(3-2) = 9$.

4.2. Casos especiais da aritmética

A linguagem trata alguns casos que, na matemática pura, não têm resposta única, de um jeito específico e previsível:

- 0^0 vale 1.
- Uma divisão por zero não interrompe o programa, o resultado vira “infinito” (com o sinal do numerador), e a execução continua.
- Um número elevado a um expoente tão grande que o resultado passa da capacidade do computador também vira “infinito”, sem parar o programa.
- Elevar um número **negativo** a um expoente **não inteiro** (por exemplo, $(-4)^{0.5}$, que seria uma raiz quadrada de negativo) é um erro fatal, porque o resultado seria um número complexo, fora do que a linguagem representa.

O capítulo 7 detalha a diferença entre um erro que para o programa e um que só substitui o valor e segue em frente.

4.3. Operadores relacionais

```
=    igual
<> diferente
<    menor
<=  menor ou igual
>    maior
>=  maior ou igual
```

Usados dentro de um IF (capítulo 6) para testar uma condição. O resultado é sempre um número: 1 se a comparação for verdadeira, 0 se for falsa.

```
100 IF X > 3 THEN 200
```

Números são comparados numericamente e cadeias são comparadas caractere a caractere, mas **cadeias só podem usar = e <>**, nunca <, <=, > ou >=. Comparar um número com uma cadeia é inválido.

4.4. Funções embutidas e definidas

Uma expressão pode chamar uma das onze funções matemáticas embutidas (capítulo 5) ou uma função criada com DEF FN (capítulo 6):

```
100 PRINT SQR(16)
110 DEF FNF(X) = X^2 + 1
120 PRINT FNF(3)
```

A linha 100 (função embutida) imprime 4; a linha 120 (função definida) imprime 10.

4.5. Expressões de cadeia

Diferente dos números, uma expressão de cadeia é bem limitada: só pode ser uma constante entre aspas ou uma variável de cadeia, **não existe operador de concatenação** nem qualquer outra operação sobre cadeias na linguagem. Se você precisa montar um texto a partir de pedaços, isso precisa ser feito pedaço por pedaço, por exemplo com vários itens num mesmo PRINT (capítulo 6).

5. Funções embutidas

A linguagem tem exatamente onze funções matemáticas prontas. Todas recebem um único argumento numérico entre parênteses com exceção de RND, a única sem argumento nenhum. Diferente das outras palavras reservadas, o nome de uma função pode vir colado ao parêntese, sem espaço: SQR(4) é válido.

Função	O que calcula
ABS(X)	Valor absoluto de X
ATN(X)	Arco-tangente de X, em radianos
COS(X)	Cosseno de X (X em radianos)
EXP(X)	e elevado a X
INT(X)	Maior inteiro não maior que X (arredonda para baixo)
LOG(X)	Logaritmo natural de X
RND	Um número pseudoaleatório entre 0 (inclusivo) e 1 (exclusivo)
SGN(X)	1 se X é positivo, -1 se negativo, 0 se X é zero
SIN(X)	Seno de X (X em radianos)
SQR(X)	Raiz quadrada de X
TAN(X)	Tangente de X (X em radianos)

Detalhes que valem atenção

INT arredonda para baixo, não para o mais próximo. INT(3.9) vale 3, e INT(-3.1) vale -4 (o maior inteiro que *não é maior* que -3.1, ou seja, mais para o lado negativo). Se você precisa arredondar para o inteiro mais próximo, precisa montar a conta você mesmo. Por exemplo, INT(X + 0.5) para valores positivos.

SQR de um número negativo é erro fatal. A linguagem não tem número complexo, então não há resultado possível. O mesmo vale para LOG de zero ou de um número negativo.

RND sem RANDOMIZE repete sempre a mesma sequência de números, toda vez que você roda o programa. Isso é proposital: ajuda a testar e depurar um programa que usa números aleatórios, já que o resultado é sempre igual de uma execução para outra. Para obter uma sequência diferente a cada execução, use o comando RANDOMIZE (capítulo 6) antes do primeiro RND.

```
100 RANDOMIZE
110 PRINT INT(RND * 6) + 1
```

Isso imprime um número de 1 a 6, como um dado.

6. Referência de comandos

Este capítulo documenta, um por um, todos os comandos da linguagem. Cada seção mostra a sintaxe, explica o que o comando faz e dá um exemplo executável.

6.1. LET — atribuição

LET variável = expressão

Guarda o valor de uma expressão numa variável (simples, de arranjo, ou de cadeia). O lado esquerdo e o lado direito precisam ser do mesmo tipo, não dá para atribuir uma cadeia a uma variável numérica, nem vice-versa.

```
100 LET X = 5
110 LET Y = X * 2 + 1
120 LET N$ = "MARIA"
130 LET A(3) = Y
```

6.2. PRINT — saída

PRINT [item] [(,|;) item]...

Imprime uma sequência de itens, expressões numéricas, cadeias, ou TAB(expressão) para posicionar o cursor numa coluna específica separados por vírgula ou ponto e vírgula.

- **Vírgula** avança o cursor para a próxima *zona de impressão* (a linha é dividida em zonas de largura fixa, como colunas de uma tabela).
- **Ponto e vírgula** não insere espaço nenhum, só separa um item do outro.
- Um PRINT sozinho, sem nada depois, imprime uma linha em branco.
- Se a lista terminar em vírgula ou ponto e vírgula, o cursor **não** pula para a próxima linha, o próximo PRINT continua de onde este parou.

```
100 PRINT "OLA, MUNDO"
110 PRINT "A =" ; A
120 PRINT "COLUNA 1", "COLUNA 2"
130 PRINT TAB(20); "ALINHADO NA COLUNA 20"
```

Na linha 110, o ponto e vírgula não insere espaço extra antes do valor de A; na linha 120, a vírgula manda cada item para uma zona de impressão diferente.

Um número impresso sempre vem com um espaço (se positivo) ou o sinal de menos (se negativo) antes dele, e um espaço depois. É assim que a norma define a formatação numérica, para que números fiquem visualmente separados uns dos outros mesmo sem separador explícito.

6.3. INPUT — entrada do usuário

INPUT variável [, variável]...

Pausa o programa, mostra o aviso ? e espera o usuário escrever um valor (ou vários, separados por vírgula) para cada variável da lista.

```
100 PRINT "QUAL O SEU NOME?"
110 INPUT N$
120 PRINT "OLA, "; N$
```

Se o usuário escrever a quantidade errada de valores, ou um texto onde um número era esperado, o programa **não** para, ele avisa e pede a resposta de novo. Essa é uma diferença importante em relação a LET e READ, onde o mesmo tipo de erro é fatal.

6.4. READ e DATA — dados constantes

```
READ variável [, variável]...  
DATA valor [, valor]...
```

DATA declara uma lista de valores constantes, fixos no código do programa. READ lê, um de cada vez, os próximos valores dessa lista e guarda nas variáveis indicadas como um INPUT automático, cujos “valores informados” já estão escritos no programa.

```
100 READ A, B, C  
110 PRINT A + B + C  
120 DATA 10, 20, 30
```

Repare que o DATA pode aparecer em qualquer linha do programa, não precisa estar perto do READ que vai consumi-lo. Todos os DATA do programa formam uma única lista, na ordem em que aparecem.

Um valor no DATA pode ser um número, uma string entre aspas, ou uma cadeia *sem* aspas (só letras, dígitos, espaços, +, - e .):

```
100 DATA 3.14159, PI, 5E-10, ", "
```

Diferente do INPUT, se o READ não encontrar dado suficiente, ou encontrar um dado do tipo errado (por exemplo, tentar ler uma cadeia para uma variável numérica), o programa **para**, é um erro fatal.

6.5. RESTORE — reiniciar a leitura

```
RESTORE
```

Volta o ponteiro de leitura do READ para o primeiro valor de DATA do programa, permitindo ler a mesma lista de novo desde o início.

```
100 DATA 1, 2, 3  
110 READ A, B, C  
120 PRINT A; B; C  
130 RESTORE  
140 READ A  
150 PRINT A
```

A última linha imprime 1 de novo, RESTORE fez o READ da linha 140 reler o primeiro valor de DATA.

6.6. DIM — declarar um array

```
DIM nome(limite) [, nome(limite)]...  
DIM nome(limite, limite) [, ...]...
```

Declara explicitamente o tamanho de um arranjo, veja a seção 3.3 para uma explicação completa. DIM precisa vir antes do primeiro uso do arranjo que ele declara, e cada arranjo só pode ser dimensionado uma vez.

```
100 DIM NOTAS(50), TABELA(10, 10)
```

6.7. OPTION BASE — mudar o índice inicial dos arrays

```
OPTION BASE 0  
OPTION BASE 1
```

Muda o índice inicial dos arranjos de 0 (o padrão) para 1. Só pode aparecer uma vez no programa, e precisa vir antes de qualquer DIM ou uso de arranjo, veja a seção 3.4.

6.8. DEF — definir uma função

```
DEF FNx = expressão  
DEF FNx(parâmetro) = expressão
```

Cria uma função com nome FNx (x é uma única letra), reutilizável em qualquer expressão do programa a partir dali. Uma função pode ter zero ou um parâmetro.

```
100 DEF FNQUAD(X) = X * X  
110 PRINT FNQUAD(5)  
120 DEF FNPI = 3.14159  
130 PRINT FNPI
```

A linha 110 imprime 25; a linha 130 imprime 3.14159.

Uma função precisa ser definida **antes** da primeira linha que a usa, e não pode chamar a si mesma (nem direta, nem indiretamente, através de outra função). A linguagem não tem recursão. O parâmetro de uma função é local à sua definição: se o parâmetro se chamar X, esse X vale só dentro do cálculo da função, sem afetar (nem ser afetado por) uma variável X usada no resto do programa.

6.9. GOTO e GOSUB — desvio incondicional

```
GOTO linha  
GOSUB linha  
RETURN
```

GOTO linha (também pode ser escrito GO TO linha, com espaço) transfere a execução direto para o número de linha indicado, sem nenhuma condição.

GOSUB linha faz a mesma coisa, mas *lembrando* de onde veio: quando o programa encontrar um RETURN, ele volta para a linha logo depois do GOSUB. É o mecanismo de sub-rotina da linguagem, útil para um trecho de cálculo repetido em vários pontos do programa.

```
100 LET A = 4 : GOTO 200  
...  
200 PRINT A
```

(O : no exemplo acima é só ilustrativo, a norma não permite dois comandos numa mesma linha; seria preciso separar em duas linhas.)

```
100 LET N = 5  
110 GOSUB 500  
120 PRINT "O DOBRO É "; R  
130 STOP  
500 REM SUB-ROTINA: DOBRA N, GUARDA EM R  
510 LET R = N * 2  
520 RETURN
```

Um RETURN sem um GOSUB correspondente (por exemplo, se o fluxo do programa “cair” na sub-rotina em vez de ser chamado por GOSUB) é um erro fatal. Por isso o exemplo acima usa STOP antes da linha 500, para não continuar direto para dentro da sub-rotina sem ter sido chamada.

6.10. IF...THEN — desvio condicional

IF condição THEN linha

Testa uma condição (uma comparação, veja a seção 4.3) e, se for verdadeira, transfere a execução para o número de linha indicado. Se for falsa, a execução simplesmente continua na linha seguinte.

```
100 INPUT X
110 IF X > 0 THEN 140
120 PRINT "NAO E POSITIVO"
130 GOTO 150
140 PRINT "E POSITIVO"
150 END
```

Não existe ELSE nem bloco IF...THEN...END IF — o THEN sempre é seguido de um número de linha (nunca de outro comando direto), e o “caminho contrário” é sempre escrito como a continuação natural do programa, como no exemplo acima.

6.11. ON...GOTO — desvio para vários lugares

ON expressão GOTO linha [, linha]...

Calcula uma expressão numérica, arredonda para o inteiro mais próximo, e usa esse número para escolher qual linha da lista visitar: 1 escolhe a primeira, 2 a segunda, e assim por diante. Se o valor calculado for menor que 1 ou maior que a quantidade de linhas na lista, é um erro fatal.

```
100 PRINT "1-VERMELHO 2-VERDE 3-AZUL"
110 INPUT N
120 ON N GOTO 200, 300, 400
130 PRINT "OPCAO INVALIDA"
140 GOTO 100
200 PRINT "VOCE ESCOLHEU VERMELHO"
210 GOTO 500
300 PRINT "VOCE ESCOLHEU VERDE"
310 GOTO 500
400 PRINT "VOCE ESCOLHEU AZUL"
500 END
```

6.12. FOR e NEXT — laço contado

FOR variável = início TO limite [STEP incremento]
... corpo do laço ...
NEXT variável

Repete o trecho entre FOR e NEXT uma vez para cada valor da variável de controle, começando em início e indo até limite (inclusive), avançando de incremento em incremento a cada volta. STEP é opcional. Se omitido, o incremento é 1.

```
100 FOR I = 1 TO 5
110 PRINT I
120 NEXT I
130 END
```

Isso imprime 1 2 3 4 5, um por linha. Um incremento negativo conta para trás:

```
100 FOR I = 10 TO 1 STEP -1
110   PRINT I
120 NEXT I
```

O início e o limite são calculados uma única vez, ao entrar no laço e não a cada volta. Mudar o valor de uma variável usada como limite, dentro do corpo do laço, não afeta quantas vezes ele repete.

Se a condição já falhar de cara (por exemplo, FOR I = 10 TO 1 sem STEP negativo), **o corpo não executa nem uma vez**. O laço é testado *antes* de cada volta, nunca depois.

Um laço FOR pode conter outro dentro dele (aninhado), desde que cada NEXT feche o FOR mais interno ainda aberto, sem entrelaçar:

```
100 FOR I = 1 TO 3
110   FOR J = 1 TO 3
120     PRINT I; J
130   NEXT J
140 NEXT I
```

Dois laços aninhados não podem usar a mesma variável de controle, e nenhum comando pode desviar (GOTO, GOSUB, IF . . . THEN) para *dentro* de um laço vindo de *fora* dele, só de dentro para fora, ou de dentro para dentro do mesmo laço.

6.13. RANDOMIZE

RANDOMIZE

Reinicia a sequência de números pseudoaleatórios usada por RND (capítulo 5) a partir de uma semente imprevisível, para que cada execução do programa produza uma sequência diferente.

6.14. STOP e END

STOP
END

STOP interrompe a execução do programa imediatamente, em qualquer ponto. Pode aparecer quantas vezes forem necessárias.

END também interrompe a execução, mas com uma regra a mais: **todo programa precisa ter exatamente um END, e ele precisa ser a última linha do programa**. Pense em END como marcando “aqui termina o texto do programa” (mesmo que a execução real pare antes, por um STOP ou um erro), e STOP como “pare de executar aqui”, sem relação com onde o texto do programa termina.

6.15. REM — comentário

REM texto livre

Marca o resto da linha como comentário: não é executado. Veja a seção 2.6.

```
100 REM ESTE PROGRAMA CALCULA A AREA DE UM CIRCULO
110 INPUT R
120 PRINT 3.14159 * R^2
```

7. Erros e exceções

Nem todo problema em tempo de execução para o programa. A norma distingue dois tipos de situação anormal, e este interpretador segue essa distinção:

7.1. Erros fatais

Interrompem o programa imediatamente, mostrando uma mensagem que indica a linha onde o problema ocorreu. Alguns exemplos:

- Ler uma variável (ou elemento de array) antes de atribuir um valor a ela (seção 3.2).
- Índice de array fora do intervalo declarado.
- SQRT de um número negativo, ou LOG de zero ou negativo.
- Base negativa elevada a um expoente não inteiro.
- RETURN sem um GOSUB correspondente.
- ON . . . GOTO com um valor fora da lista de linhas.
- GOTO, GOSUB ou IF . . . THEN apontando para um número de linha que não existe no programa.
- Atribuir (LET ou READ) uma string maior que 18 caracteres a uma variável de string.
- Usar a mesma letra como variável simples e como array.
- Uma função (DEF FN) chamando a si mesma.

A maioria dessas checagens acontece **antes** de qualquer linha do programa rodar. O interpretador analisa a estrutura do programa inteiro primeiro (por exemplo, conferindo que todo GOTO aponta para uma linha que existe de verdade) e só começa a executar se tudo estiver consistente. Isso significa que, para esses casos, o programa nem chega a imprimir nada antes de ser rejeitado.

7.2. Situações não fatais

Nem toda situação incomum interrompe o programa, algumas têm uma recuperação definida, e a execução continua normalmente:

- **Divisão por zero** não para o programa, o resultado vira “infinito” (positivo ou negativo, conforme o sinal do numerador).
- **Um cálculo grande demais** (por exemplo, $9999 \wedge 9999$) também vira “infinito”, sem interromper nada, mas se esse “infinito” for usado depois como índice de array, aí sim vira erro fatal (índice fora do intervalo).
- $0 \wedge 0$ **não é erro**, o resultado é definido como 1.
- No INPUT, escrever a quantidade errada de valores, ou um texto onde um número era esperado, não para o programa: o interpretador avisa e pede a resposta de novo.
- Um argumento inválido de TAB (por exemplo, um número menor que 1) é ajustado automaticamente, sem interromper o programa.

7.3. Pontos que a norma deixa em aberto

A norma ECMA-55 define, deliberadamente, alguns comportamentos como “a critério de cada implementação” (formalmente, *implementation-defined* — Apêndice 4 da norma), desde que respeitados certos limites mínimos. Este interpretador toma decisões específicas nesses pontos, documentadas em detalhe nos comentários de `src/interpreter.hpp`. Resumidamente:

Ponto	Decisão deste interpretador
Variável não inicializada	Erro fatal (a norma recomenda isso, mas não exige)
Faixa de overflow/underflow	A faixa nativa do <code>double</code> ($\sim 1\text{E}\pm 308$)
Zonas de impressão do <code>PRINT</code>	5 zonas de 16 colunas (margem de 80)
Precisão de exibição de números	8 dígitos significativos
Prompt do <code>INPUT</code>	?
<code>RND</code> sem <code>RANDOMIZE</code>	Sempre a mesma sequência

Essas escolhas foram calibradas comparando a saída deste interpretador com a do **bas55** (Jorge Giner Cordero), uma implementação madura e testada há anos contra a bateria de conformidade da NBS (ver `tests/nbs/README.md`).

8. O ambiente interativo

Rodar `ecma55basic` sem nenhum argumento abre o ambiente interativo: um espaço para escrever, testar e ajustar um programa aos poucos, em vez de escrever tudo de uma vez num arquivo.

```
$ ecma55basic
ECMA-55 BASIC [AM-42] - v0.1.0.2609.07
(c) 2016-2018 (2026 - Revised)
Augusto Manzano - All rights reserved
Type HELP to see the environment commands.
]
```

O `]` é o “prompt”: sinal de que o ambiente está esperando você escrever algo.

Importante: os comandos deste capítulo (`RUN`, `LIST`, `NEW`, `LOAD`, `SAVE`, `LOCALIZE`, `SYSTEM`, `HELP`) são ferramentas do *ambiente*, não fazem parte da linguagem BASIC em si, a norma ECMA-55 não os define. São convenções universais entre praticamente todo BASIC interativo da época (o mesmo padrão do Dartmouth BASIC, do GW-BASIC, etc.).

8.1. Escrevendo um programa

Uma linha que começa com número entra (ou substitui) uma linha do programa guardado na memória do ambiente:

```
] 10 PRINT "OLA, MUNDO"
] 20 END
```

Para apagar uma linha, escreva só o número dela, sem nada depois:

```
] 10
```

8.2. RUN, LIST, NEW

```
] RUN      roda o programa escrito até aqui
] LIST     mostra o programa escrito até aqui
] NEW      apaga o programa da memória, para começar outro do zero
```

8.3. Modo imediato

Uma linha escrita **sem** número na frente roda na hora, como se fosse uma calculadora:

```
] PRINT 2 + 2
4
] LET X = 10
] PRINT X * 3
30
```

Isso é útil para testar uma expressão ou conferir o valor de uma variável rapidamente, sem precisar montar um programa inteiro.

Atenção: o modo imediato usa seu próprio espaço de variáveis, separado do que um `RUN` usa. Depois de rodar um programa com `RUN`, as variáveis dele não aparecem no modo imediato, e vice-versa. Um comando de desvio (`GOTO`, `FOR`, etc.) escrito em modo imediato não funciona, só faz sentido dentro de um programa numerado.

8.4. LOAD e SAVE

```
] SAVE "meu-programa.bas"    grava o programa atual num arquivo
] LOAD "meu-programa.bas"    lê um programa de um arquivo
```

O arquivo salvo é um .bas comum, que também pode ser rodado direto pela linha de comando (ecma55basic meu-programa.bas), sem precisar abrir o ambiente.

8.5. LOCALIZE — trocando o idioma do ambiente

```
] LOCALIZE PT    mensagens do ambiente em português
] LOCALIZE EN    mensagens do ambiente em inglês (padrão)
```

Isso afeta o banner, o HELP e **todas** as mensagens que o interpretador escreve, inclusive as mensagens de erro do lexer, do parser e do interpretador (ex.: “variável usada antes de receber um valor”). O idioma escolhido vale para a sessão inteira, tanto no modo imediato quanto num RUN.

O idioma inicial também pode ser escolhido na linha de comando, antes mesmo de abrir o ambiente:

```
ecma55basic --pt    abre o ambiente já em português
ecma55basic --en    abre o ambiente já em inglês (o padrão, mesmo sem a flag)
```

8.6. Saindo do ambiente

```
] SYSTEM
```

BYE e EXIT funcionam como sinônimos de SYSTEM. Fechar a entrada padrão (Ctrl-D no Linux/macOS, Ctrl-Z no Windows) também encerra o ambiente.

8.7. HELP

```
] HELP
```

Mostra um resumo rápido de todos os comandos deste capítulo, direto no ambiente, útil quando você não tem este manual à mão.

HELP seguido do nome de um comando ou função da linguagem mostra a descrição, a sintaxe e um exemplo, a mesma informação dos capítulos 5 e 6, condensada para o terminal:

```
] HELP PRINT
PRINT - Imprime uma lista de itens (numeros, strings, ou TAB(n)),
separados por virgula (pula pra proxima zona) ou ponto-e-virgula
(sem espaco). Sozinho, imprime uma linha em branco.
```

Sintaxe:
PRINT [item] [(,|;) item]...

Exemplo:
100 PRINT "X ="; X
110 PRINT "A", "B", "C"

Funciona para qualquer um dos 20 comandos e das 11 funções. Por exemplo, HELP SQR, HELP FOR, HELP DEF.